

---

# LECTURE 4:

# Agent Communication

---

**Introduction to Multi-Agent Systems (MESIIA, MIA)**

**Department of Computer Science and Mathematics**

**Rovira i Virgili University**

**Fall 2019**

---

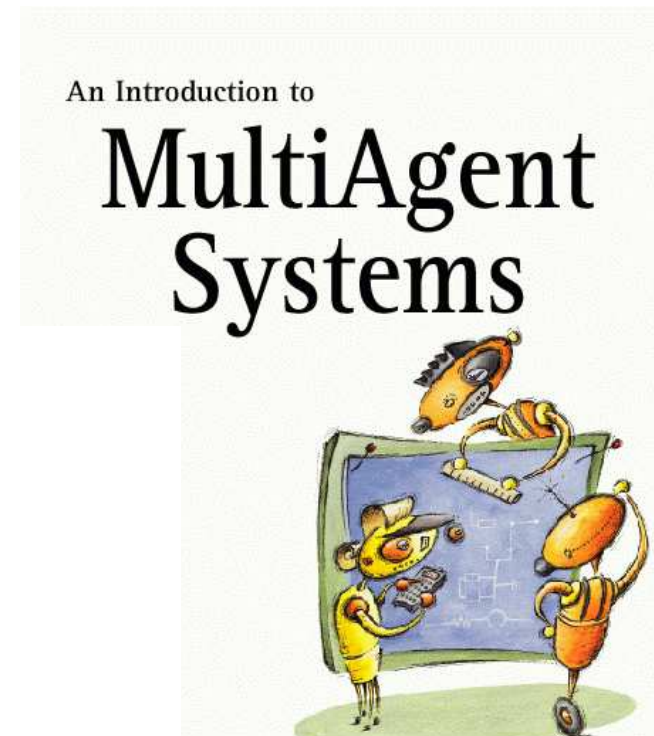
# Multi-agent systems course (I)

- First part of the course: **agents**
    - Introduction to agent technology
    - Agent **architectures**
      - Reactive / Deliberative / Hybrid
    - Agent **properties**
      - Autonomy / Learning / Reactivity / Mobility / Reasoning / Proactivity / **Social ability** / ...
    - Agent **types**
      - **Cooperative** / Interface / Information / Mobile / Reactive / Hybrid / Heterogeneous systems
-

---

# Multi-agent systems course (II)

- Second part of the course: Multi-Agent Systems
  - ❑ Agent communication
  - ❑ Coordination
  - ❑ Negotiation
  - ❑ Distributed planning
  - ❑ Coalition formation
  - ❑ Application domains



---

# Overview of today's lecture

- Collaborative agents
  - Distributed problem solving
  - Agent communication
    - Blackboard systems
    - Message passing
      - FIPA standards
      - Basic communication protocols
        - Contract Net
      - Ontologies
-

# Collaborative Agents

- A **multi-agent system** (MAS) may be seen as a collection of deliberative, autonomous, collaborative agents, that may **communicate** and **cooperate** with each other to solve complex distributed problems
- They are specially useful to solve problems with **distributed data/expertise**.



# Benefits of Multi-Agent Systems (I)

## ■ Modularity

- ❑ Each agent is specialised in the solution of a particular kind of problems (leading also to **reusability**)
- ❑ The complexity of the construction of agents is reduced
- ❑ The process of solving a complex problem is reduced to solving easier subproblems



---

# Benefits of Multi-Agent Systems (II)

## ■ Efficiency

- ❑ Problems can be solved more quickly, due to the inherent concurrency/parallelism
  - ❑ Different agents are working at the same time in different parts of a problem
    - These subproblems can be **independent** or (slightly) **dependent**
      - ❑ Share partial results
      - ❑ Coordinate the use of shared resources
-

# Benefits of Multi-Agent Systems (III)

## ■ Reliability

- ❑ Avoid single point of failure in centralised systems
- ❑ We can have redundancy
  - Different agents of the same type
  - Different agents that can do a certain task
- ❑ If an individual agent fails, the other agents can take its work and re-distribute it dynamically





---

# Benefits of Multi-Agent Systems (IV)

## ■ Flexibility

- ❑ Agents can be created/deleted dynamically, depending on the amount of work to be done, the available resources, etc
  - ❑ Agents can dynamically generate subtasks and look for helping agents
  - ❑ Agents with different skills may dynamically form teams/coalitions to work together
-

# Distributed problem solving

- Agents **work together** to solve problems that require collective effort
- Requires:
  - **Coherence**
    - Need to *want* to work together
  - **Competence**
    - Need to know *how* to work together
- **Distributed Planning**
  - *Distributed Problem Solving* in which the agents work together **to construct a plan**



---

# Distributed problem solving steps

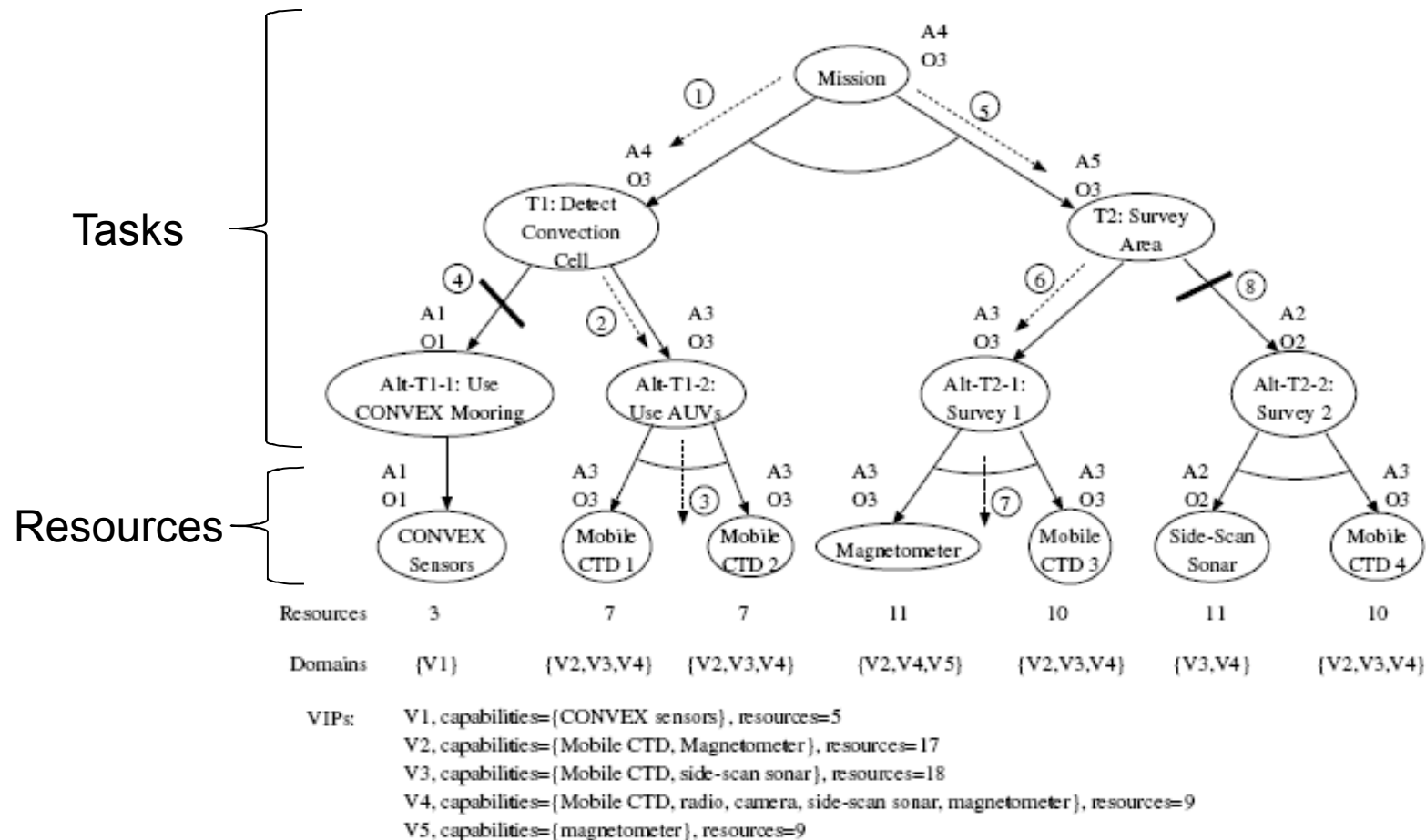
1. Task Decomposition
  2. Task Allocation
  3. Task Accomplishment
  4. Result Synthesis
-

---

# 1-Task decomposition

- **Divide** a complex problem into a set of tasks (e.g. a **set of subproblems**)
  - **Decompose** (*recursively*, if necessary) large tasks into subtasks that can be tackled by different agents
  - Tasks can be totally **independent** or can have some **relationships**
  - This decomposition may be done by the *user*, by a *central coordinator agent* or may be negotiated at run time by *all the agents* of the system
-

# Example of hierarchical task decomposition in an AND-OR tree



## 2-Task allocation

- **Assign** the subproblems to different agents
- **Easy case**: there is only one agent capable of solving each subproblem
  - No flexibility
- **Interesting case**: the same subproblem can be solved by different kinds of agents
  - Dynamic mechanisms of run-time task allocation (e.g. Contract Net)
  - Possibility of recovering from individual failures



---

## 3-Task accomplishment

- Each agent solves (on its own) its assigned subproblems
- This phase can be done, in principle, **without communication** (if the problems are independent)
- There may be **dependencies** between subproblems
  - Potential **conflicts** may appear



# Conflict management



- Examples
  - Two sub-solutions are incompatible
  - Conflicts in the **use of shared resources**
- Agents have to communicate with each other to solve these situations
- There may be different solutions to the same subproblem



---

# Agent support in task execution (I)

## ■ Task sharing

- An agent can **request the help of other agents** to solve a particular task
    - Too complex / expensive for the agent to do individually
    - It can know that other agents have the appropriate knowledge/skills to solve that task
    - It can know that other agents already have to solve that task
  - Problem of *task assignment*
    - Who can I ask for help?
    - How do I know what tasks can other agents do?
-

---

# Agent support in task execution (II)

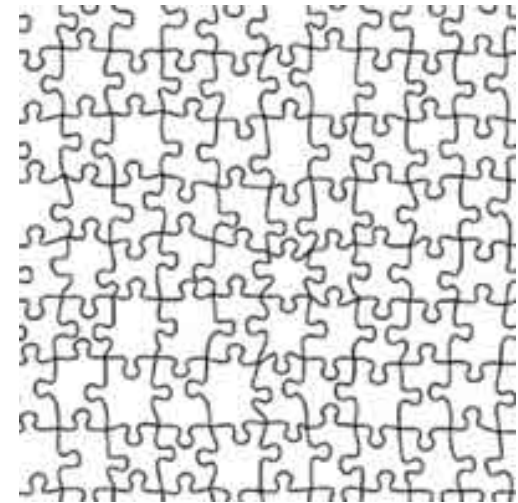
## ■ Result sharing

- Use **intermediate results** obtained by other agents
    - [ For example, shortest routes to certain points of the map ]
  - Agents can provide intermediate subsolutions to help other agents in their work
  - That allows a fast recognition of
    - **Incorrect solutions**
      - An agent, on the basis of its knowledge, can detect an error on the **results** of other agents
    - **Conflictive solutions**
      - An agent can detect possible conflicts between its results and **subsolutions** of other agents
  - Cooperation/Negotiation to solve these problems
-

---

## 4-Result synthesis

- Put together the results of all agents to find the complete solution
- Who makes it?
- How is it made?
- If each subproblem has a unique solution, it is a relatively easy step
- Otherwise, there may be need of conflict detection, task re-assignment, ...



---

## Summary: questions about the design of a collaborative MAS (I)

- *Who* decomposes the problem?
    - One agent / all agents / user
  - *How* is the problem decomposed?
    - 1 level / n levels
    - Static – beginning / Dynamic – execution time
  - *Who assigns tasks* to agents? How is the assignment made?
    - Static – beginning / Dynamic – execution time
-

---

## Summary: questions about the design of a collaborative MAS (II)

- What kind of *interactions* can exist between agents when they are solving tasks?
    - Conflict detection
    - Conflict resolution
  - How are *tasks shared*?
  - How are *results shared*?
  - Who/how *merges* the subresults to get the final result?
-

---

# Why do we need Agent Communication?

- Multi-agent systems allow distributed problem solving
- This requires the agents to coordinate their actions
- Agent communication facilitates this by allowing individual agents to interact
  - allows cooperation
  - allows information sharing



---

# Communication Categories

- We can classify communication in a **mechanistic manner**
    - via the type of **sendee-addressee link**
    - via the **nature of the medium**
  - or in a higher level **meaning-based manner**
    - via the **type of intention**
-

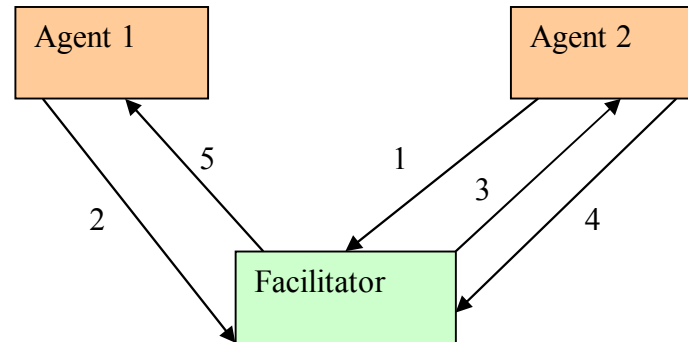
---

# The Sendee - Addressee Link

- Communication can be
    - Point to Point
      - An agent talks directly to another agent
    - Broadcast
      - An agent sends some information to a group of agents
    - Mediated
      - The communication between two agents is mediated by a third party
      - Example: **facilitators**
-

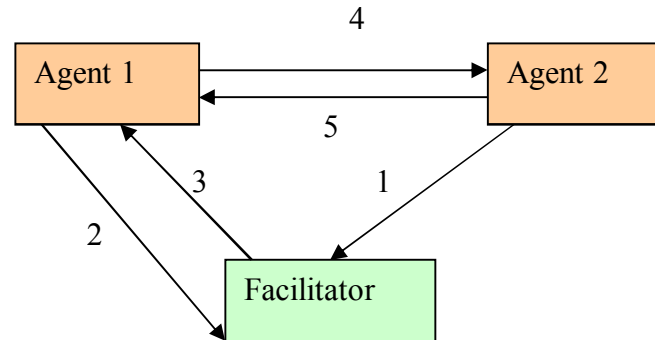


# Communication via facilitator (I)



1. Agent\_2 tells the facilitator the services it provides
2. Agent\_1 asks to the facilitator who can provide a certain service with some conditions
3. The facilitator requests the service to agent\_2
4. Agent\_2 provides the answer
5. The facilitator sends the answer to agent\_1

# Communication via facilitator (II)



1. Agent\_2 tells the facilitator the services it provides
2. Agent\_1 asks to the facilitator who can provide a certain service with some conditions
3. The facilitator tells agent\_1 that agent\_2 can do that service
4. Agent\_1 requests the service from agent\_2
5. Agent\_2 sends the answer to agent\_1

---

# Locating other agents

- Unless we use some broadcast techniques (e.g. blackboard systems), agents must know the addresses of other agents - possible solutions are
  - Complete internal directory
  - Partial/hierarchical internal directory
  - Mediated (e.g. JADE's DF)



# JADE – Directory Facilitator

Jadex Control Center 0.96 (2007/06/01): Project Apps2

File Refresh Help

lars

df@lars nio-mtp://syssta

### Registered Agent Descriptions

| Agent           | Leasetime | Services        | Ontologies | Languages | Protocols |
|-----------------|-----------|-----------------|------------|-----------|-----------|
| Carry_0@lars    | n/a       | service_carry   |            |           |           |
| Carry_1@lars    | n/a       | service_carry   |            |           |           |
| Carry_2@lars    | n/a       | service_carry   |            |           |           |
| Production_3... | n/a       | service_produce |            |           |           |
| Production_4... | n/a       | service_produce |            |           |           |
| Sentry_5@lars   | n/a       | service_sentry  |            |           |           |

### Registered Services

| Name            | Type            | Ownership             | Agent          | Onto... | Lan... | Prot... | Prop... |
|-----------------|-----------------|-----------------------|----------------|---------|--------|---------|---------|
| service_carry   | service_carry   | University of Hamburg | Carry_0@lars   |         |        |         |         |
| service_carry   | service_carry   | University of Hamburg | Carry_1@lars   |         |        |         |         |
| service_carry   | service_carry   | University of Hamburg | Carry_2@lars   |         |        |         |         |
| service_produce | service_produce | University of Hamburg | Production_... |         |        |         |         |
| service_produce | service_produce | University of Hamburg | Production_... |         |        |         |         |
| service_sentry  | service_sentry  | University of Hamburg | Sentry_5@la... |         |        |         |         |

### Service Properties

Name: service\_carry

Type: service\_carry

Ownership: University of Hamburg

Agent: Carry\_2@lars

| Ontologies | Languages | Protocols | Properties |
|------------|-----------|-----------|------------|
|            |           |           |            |

# Nature of the medium

## ■ Direct routing

- ❑ Message sent directly to other agent(s) with no interception or attenuation in strength

## ■ Signal propagation routing

- ❑ Commonly used by reactive agents
- ❑ Agent sends signal whose intensity decreases according to distance (e.g. radio signal in physical robots)

## ■ Public notice routing

- ❑ Blackboard systems

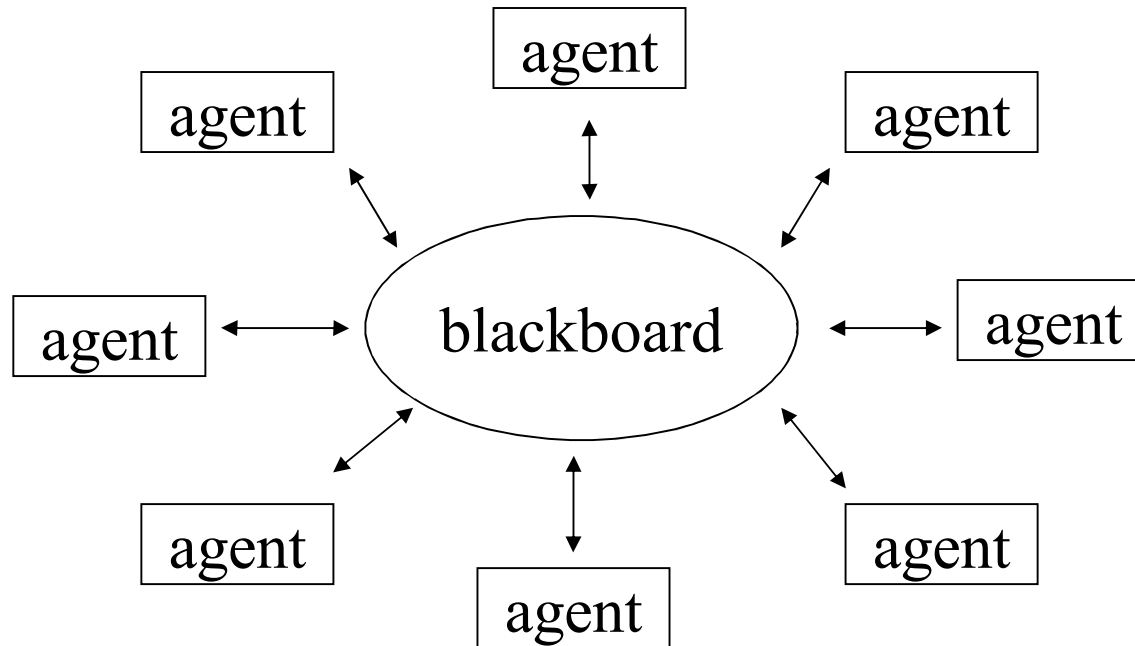


---

# Agent communication

- Basic options used in MAS
    - Blackboard systems
    - Direct message passing
-

# Blackboard Systems



---

# Blackboard-based communication

- Each agent can **put** information/data/knowledge on the common information space
  - Each agent can **read** from the blackboard at any moment
  - There is **no direct communication** between agents
-



---

# Information in blackboard

- Data of the common problem
  - Current state of the solution
  - Next subproblems to be solved
  - Requests of help
  - Present task of each agent
  - Intermediate results
-

---

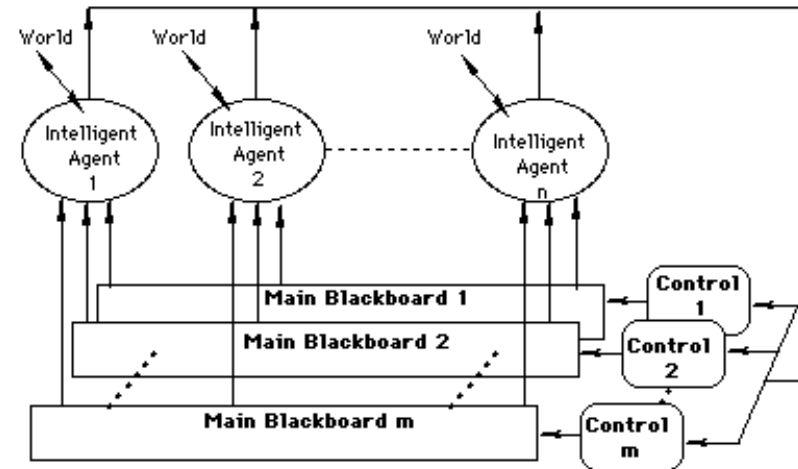
# Uses of blackboard

- ❑ Detect **conflicts**
    - Different agents that want to perform the same task
  - ❑ Notice **incompatible solutions**
    - Solutions using a shared resource at the same time
  - ❑ **Share results**
    - Agents can use partial/complete results obtained by other agents
  - ❑ **Share tasks**
    - Agents can request help in solving sub-tasks
-

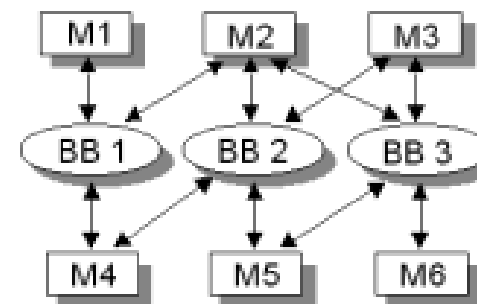
# Blackboard systems summary (I)

## ■ Positive aspects

- Flexible mechanism that allows communication/cooperation
  - E.g.  $n$  blackboards
- Independent of cooperation strategy
- It does not place any restriction on the agents' internal architecture



Multi-Blackboard Architecture

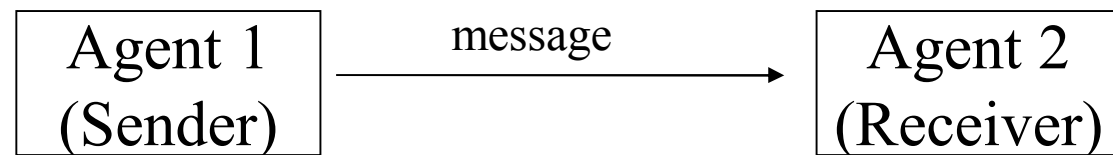


# Blackboard systems summary (II)

- Negative aspects
  - ❑ Centralised structure
  - ❑ System bottleneck
  - ❑ Everyone has to write info on the blackboard
  - ❑ Everyone has to read from the blackboard
  - ❑ Single point of failure



# Message passing



- Information is passed from one agent to another. The nature of this information can be very varied. **Speech acts** provide one way to describe this variety

# Speech Acts

- Most treatments of communication in multi-agent systems borrow their inspiration from *speech act theory*
- Speech act theories are *pragmatic* theories of language, i.e., theories of language use: they attempt to account for *how language is used by people every day to achieve their goals and intentions*
- The origin of speech act theories is usually traced to Austin's 1962 book, *How to Do Things with Words*



---

# Speech Act Theory

- Austin noticed that some utterances are like ‘physical actions’, in the sense that they appear to *change the state of the world*
  - Paradigmatic examples would be:
    - Declaring war
    - ‘I now pronounce you man and wife’
  - But more generally, *everything* we say is uttered with the intention of satisfying some goal
  - A theory of how utterances are used to achieve intentions is a **speech act theory**
-

---

# Types of speech act

- *inform* other agents about some data
  - *query* others about their current situation
  - *answer* questions
  - *request* others to act
  - *promise* to do something
  - *offer* deals
  - *acknowledge* offers and requests
  - ...
-



---

# Speech Act components

- In general, a speech act can be seen to have two components:
    - a *performative verb*:  
(e.g., request, inform, promise, ... )
    - *propositional content*:  
(e.g., “the door is open”)
-

---

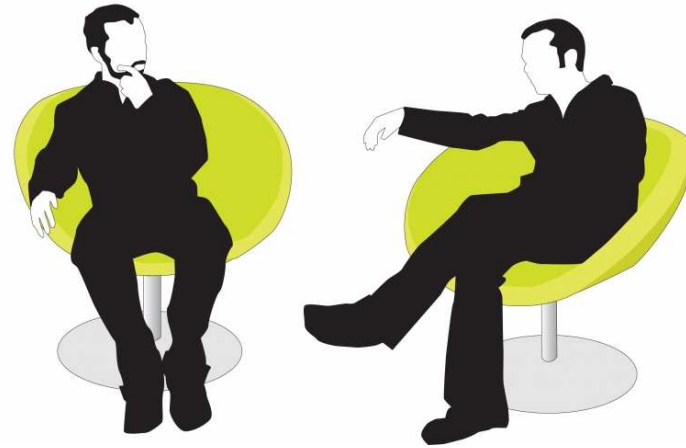
# Communicative Acts

|                                             |                          |
|---------------------------------------------|--------------------------|
| ■ Is the door open?                         | query                    |
| ■ Open the door (for me)                    | request                  |
| ■ OK! I'll open the door                    | agree                    |
| ■ The door is open                          | inform                   |
| ■ I am unable to open the door              | failure                  |
| ■ I don't want to open the door             | refuse                   |
| ■ Tell me when the door becomes open        | subscribe                |
| ■ Does anyone want to open the door?        | CFP (call for proposals) |
| ■ I can open the door for you... at a price | propose                  |
| ■ Door? What's that? I don't understand...  | not-understood           |

**Same content -open(door)- of the message, but different meanings (depending on the used performative)**

---

# Conversations



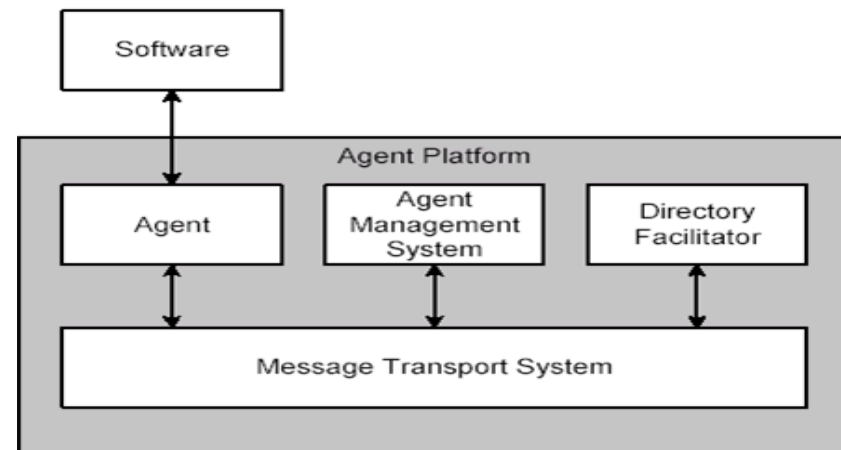
- Speech acts are rarely carried out in isolation
- Speech acts must be considered in terms of the other speech acts of the **conversation**
- Speech acts therefore can have different meanings based on the previous discourse
- Establishing meaning is thus more than understanding individual utterances

---

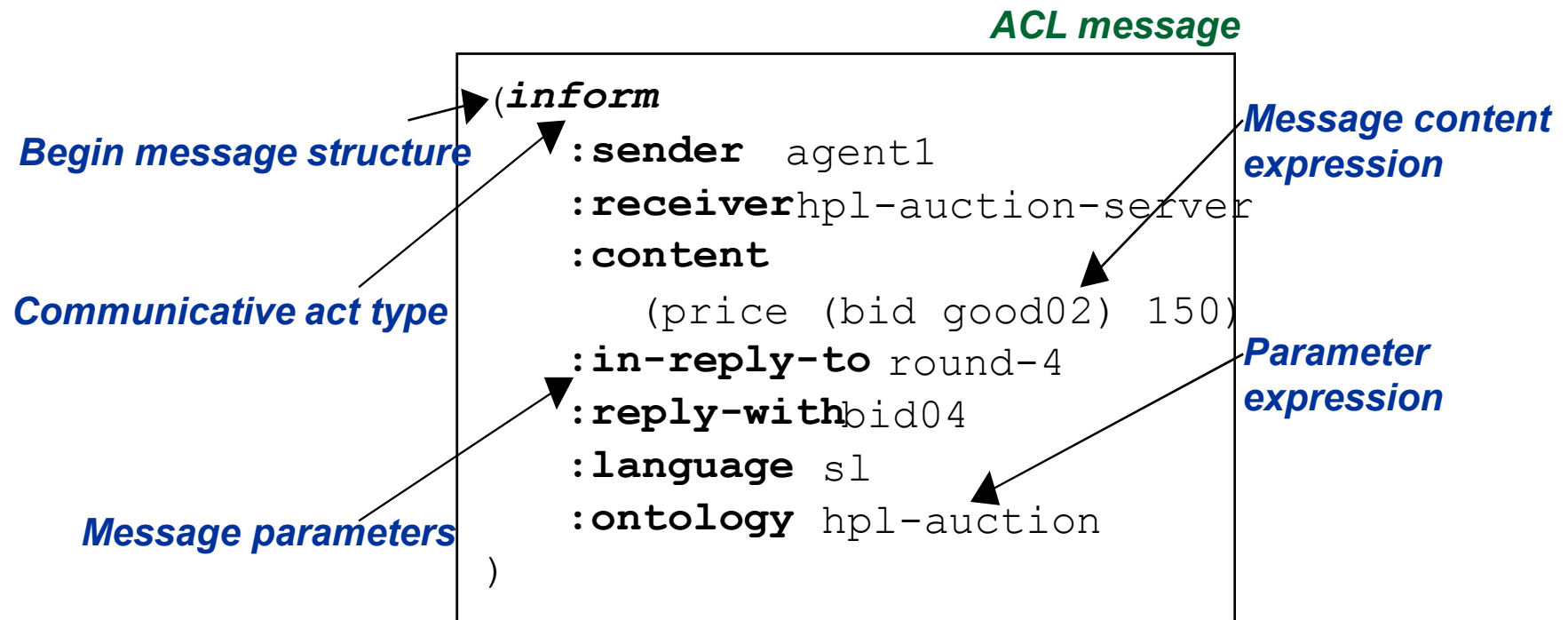
# Communication Standards

- Agents must understand each other even if running on different machines and/or different operating systems
  - Standards
    - allow different groups to write cooperating agents
    - help abstract out communication, by defining high-level general languages and protocols
-

- Foundation for Intelligent Physical Agents
- Beginning (1996): stand-alone non-profit organisation
- Now: IEEE Computer Society standards committee
- Mission: develop and promote agent standards
  - MAS architecture
  - Agent communication language (FIPA-ACL)
  - Communication protocols



# Components of a FIPA-ACL message



---

# Parameters in a FIPA ACL message

- **:sender** - who sends the message
  - **:receiver** - who is the recipient of the message
  - **:content** - content of the message
  - **:reply-with** - identifier of the message
  - **:reply-by** - deadline for replying the message
  - **:in-reply-to** - identifier of the message being replied
  - **:language** – language in which the content is written
  - **:ontology** - ontology used to represent the domain concepts
  - **:protocol** - communication protocol to be followed
  - **:conversation-id** - identifier of conversation
-

# FIPA-ACL performatives

| performative     | passing<br>info | requesting<br>info | negotiation | performing<br>actions | error<br>handling |
|------------------|-----------------|--------------------|-------------|-----------------------|-------------------|
| accept-proposal  |                 |                    | x           |                       |                   |
| agree            |                 |                    |             | x                     |                   |
| cancel           |                 | x                  |             | x                     |                   |
| cfp              |                 |                    | x           |                       |                   |
| confirm          | x               |                    |             |                       |                   |
| disconfirm       | x               |                    |             |                       |                   |
| failure          |                 |                    |             |                       | x                 |
| inform           | x               |                    |             |                       |                   |
| inform-if        | x               |                    |             |                       |                   |
| inform-ref       | x               |                    |             |                       |                   |
| not-understood   |                 |                    |             |                       | x                 |
| propose          |                 |                    | x           |                       |                   |
| query-if         |                 | x                  |             |                       |                   |
| query-ref        |                 | x                  |             |                       |                   |
| refuse           |                 |                    |             | x                     |                   |
| reject-proposal  |                 |                    | x           |                       |                   |
| request          |                 |                    |             | x                     |                   |
| request-when     |                 |                    |             | x                     |                   |
| request-whenever |                 |                    |             | x                     |                   |
| subscribe        |                 | x                  |             |                       |                   |



---

# Inform

- Content: **statement**
  - The sender **informs** the receiver that a given proposition is true
  - The sending agent
    - holds that some proposition is true
    - intends that the receiving agent also comes to believe that the proposition is true
    - does not already believe that the receiver has any knowledge of the truth of the proposition
-

---

# Inform example

```
(inform
  :sender      (agent-identifier :name i)
  :receiver    (agent-identifier :name j)
  :content     "door( now, open )"
  :language    Prolog)
)
```



---

# Query-if

- Content: **proposition**
  - The sender **asks** the receiver if a given proposition is true
  - The sending agent
    - does not know if the proposition is true
    - believes that the receiver knows if the proposition is true
-

---

# Query-if example

```
(query-if
  :sender (agent-identifier :name i)
  :receiver (set (agent-identifier :name j))
  :content
    "((registered (server d1) (agent j)))"
  :reply-with r09
  ...)
```

```
(inform
  :sender (agent-identifier :name j)
  :receiver (set (agent-identifier :name i))
  :content
    "((not (registered (server d1) (agent j))))"
  :in-reply-to r09)
```

---

---

# Query-ref

- Content: **descriptor**
  - The sender **asks** the receiver for the object referred to by the descriptor
  - The sending agent
    - does not know which object (or set of objects) corresponds to the descriptor
    - believes that the receiver knows about the objects corresponding to the descriptor
-

# Query-ref example

```
(query-ref
  :sender (agent-identifier :name i)
  :receiver (set (agent-identifier :name j))
  :content
    "((all ?x (available-service j ?x)))"
  ...)
```

```
(inform
  :sender (agent-identifier :name j)
  :receiver (set (agent-identifier :name i))
  :content
    "((= (all ?x (available-service j ?x))
      (set (reserve-ticket train)
            (reserve-ticket plane)
            (reserve automobile))))"
  ...)
```

---

# Request

- Content: **action**
  - The sender **requests** the receiver to perform some action
  - The sending agent
    - intends the action to be performed
    - believes recipient is capable of performing this action
    - does not believe that receiver already intends to perform action
-

---

# Request example

```
(request
  :sender    (agent-identifier :name i)
  :receiver  (agent-identifier :name j)
  :content   (action (agent-identifier :name j)
                  open_the_door)
  :language  fipa-sl
)
```

---

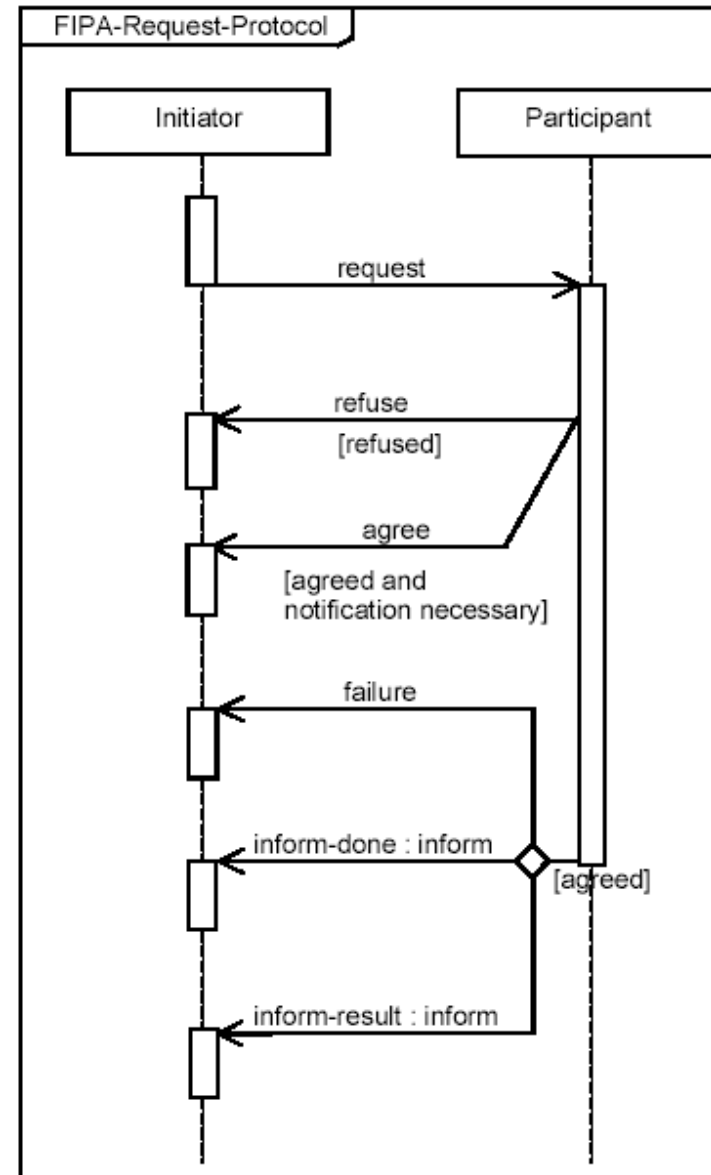


---

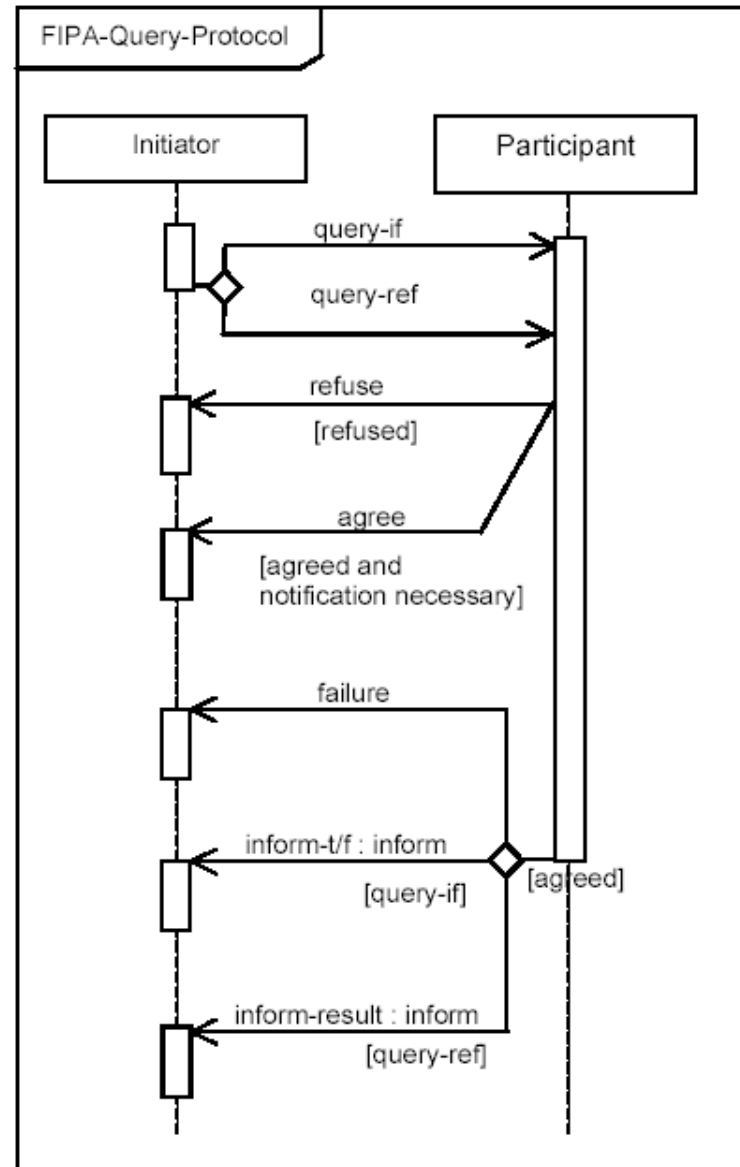
# Communication protocols

- There are many situations in which agents engaged in a dialogue with a certain purpose exchange the same sequence of messages
    - When an agent **makes a question** to another
    - When an agent **requests a service** from another
    - When an agent **looks for help** from other agents
  - To ease the management of this typical message interchanges we can use predefined *protocols*
-

# FIPA-Request protocol



# FIPA-Query protocol



---

# The Contract Net protocol

- A *task-sharing* protocol:

1. Recognition
2. Announcement
3. Bidding
4. Awarding
5. Expediting

# 1-Recognition

- In this stage, an agent recognizes it has a problem it wants help with
- An agent has a goal, and either...
  - ❑ realizes it **cannot** achieve the goal in isolation — does not have capability
  - ❑ realizes it would **prefer** not to try to achieve the goal in isolation (typically because of solution quality, deadline, use of resources, etc.)



---

## 2-Announcement

- In this stage, the agent with the task sends out an *announcement* which includes a *specification* of the task to be achieved
- Specification must encode:
  - Description of the task itself
  - Any constraints (e.g. deadlines, quality constraints)
  - Meta-task information (e.g. preference on attributes)
- The announcement is then *broadcast*



## 3-Bidding

- Agents that receive the announcement decide for themselves whether they wish to *bid* for the task
- Factors:
  - agent must decide whether it is capable of expediting task
  - agent must evaluate the cost of making the task and the benefits it can get from making it
  - agents must take into account other previous tasks that have not yet been assigned, or other simultaneous announcements ...
- If an agent chooses to bid, then it submits a *tender*, detailing the conditions on which it can execute the task



## 4-Awarding

- The agent that sent the task announcement must choose between bids & decide who to “award the contract” to
- The result of this process is communicated to the agents that submitted a bid





---

## 5-Expediting

- The successful *contractor* then expedites the task
    - That may involve generating further manager-contractor relationships: *sub-contracting*
-

---

# Contract Net

- The collection of nodes is the “contract net”
  - Each node on the network can, at different times or for different tasks, be a manager or a contractor
  - When a node gets a composite task (or for any reason it can't solve its present task), it breaks it into subtasks (if possible) and announces them (acting as a manager), receives bids from potential contractors, and then awards the job
-

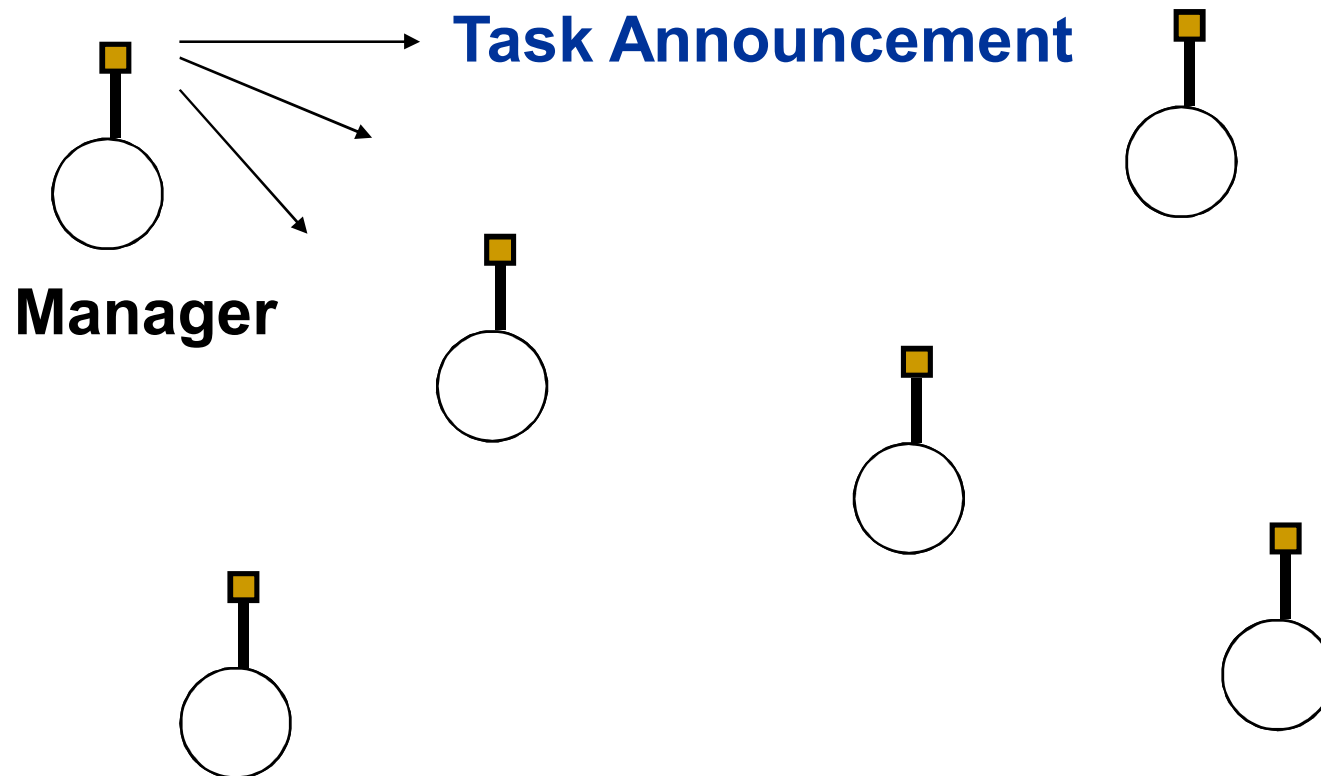
---

# Issues for Implementing Contract Net

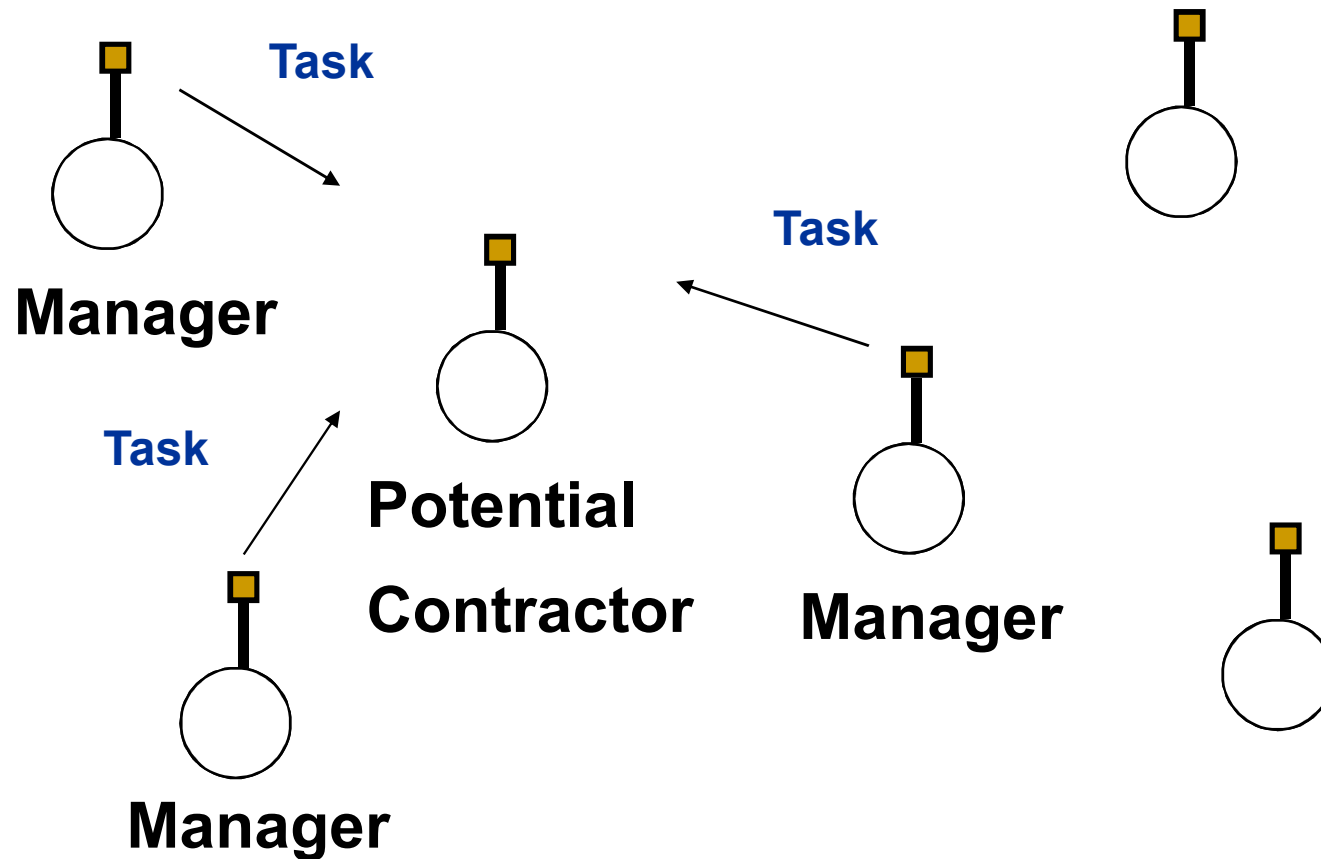
## ■ How to...

- ❑ ... specify *tasks*?
  - ❑ ... specify *quality of service*?
  - ❑ ... select between competing offers?
  - ❑ ... differentiate between offers based on multiple criteria?
-

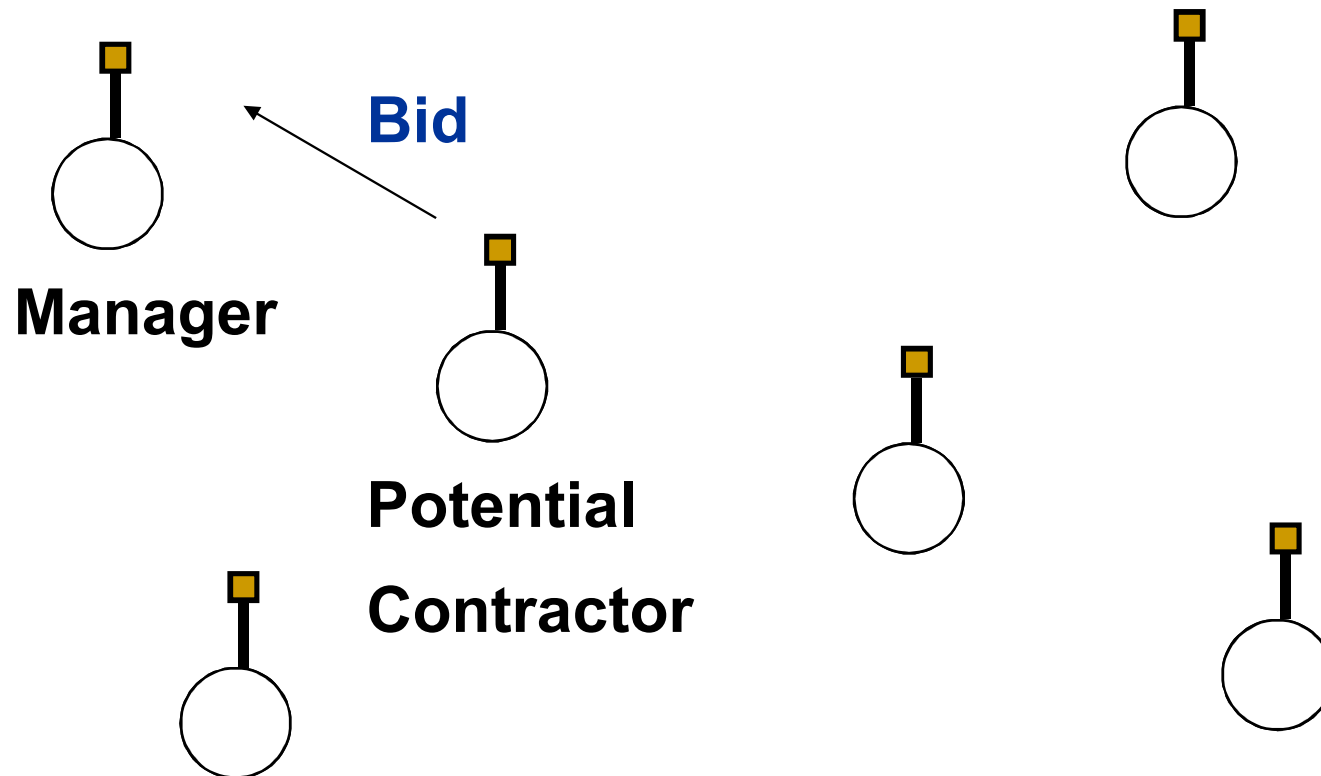
# Node Issues Task Announcement



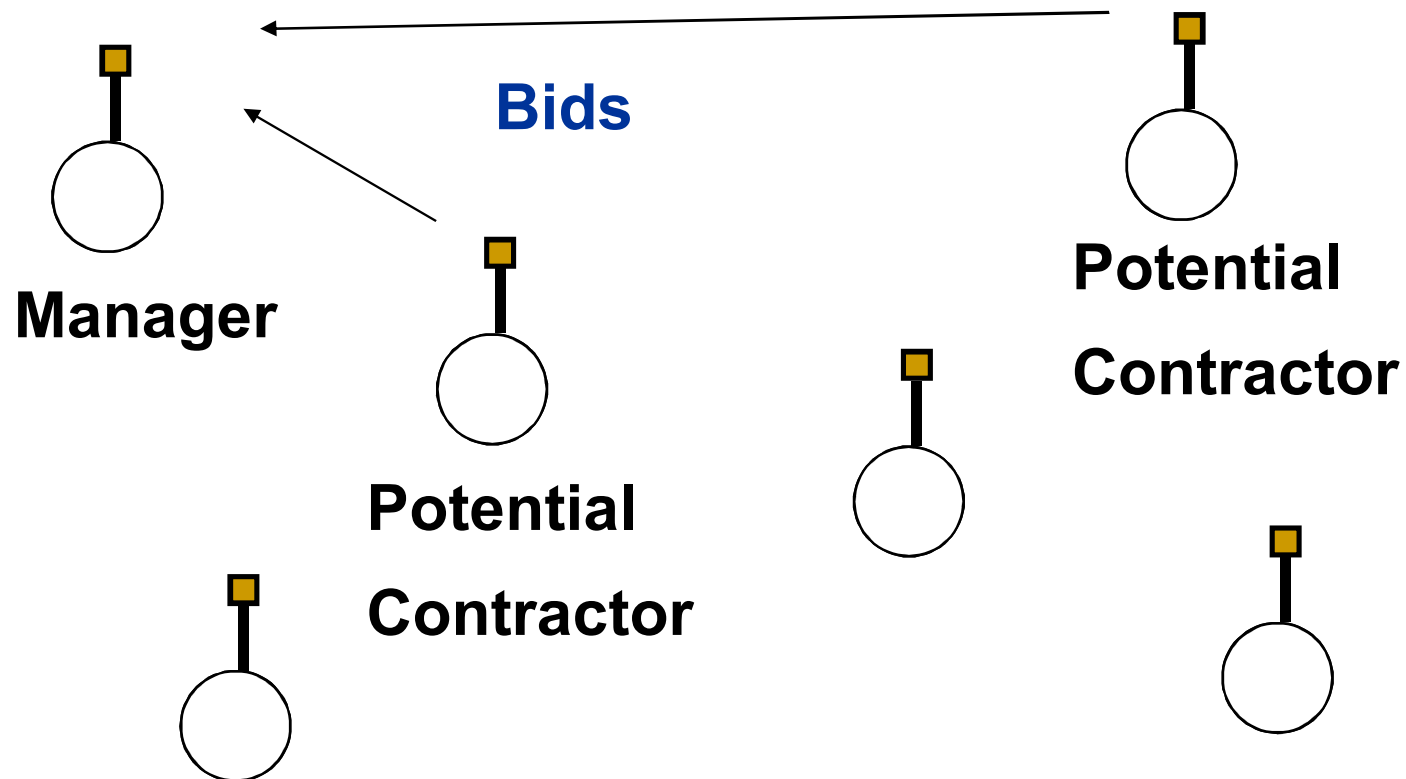
# Idle Node Listening to Task Announcements



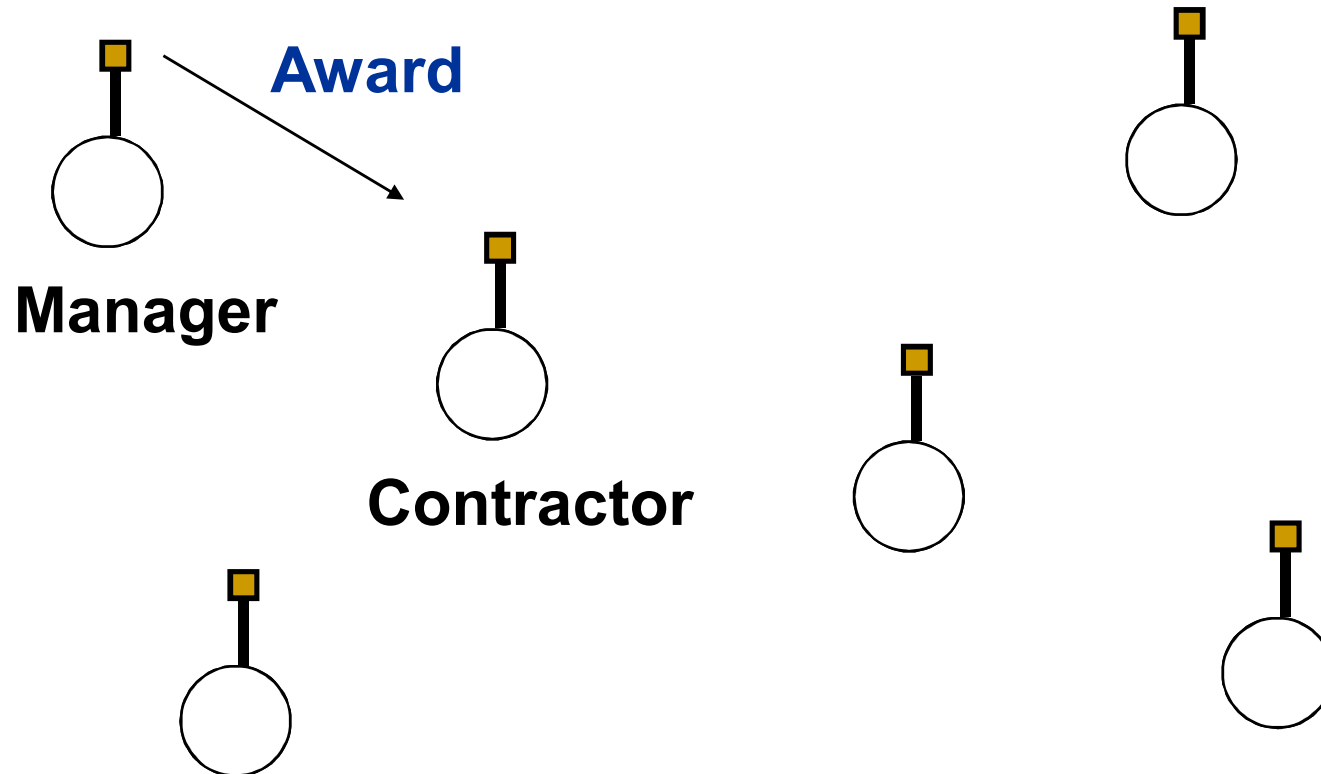
# Node Submitting a Bid



# Manager listening to bids

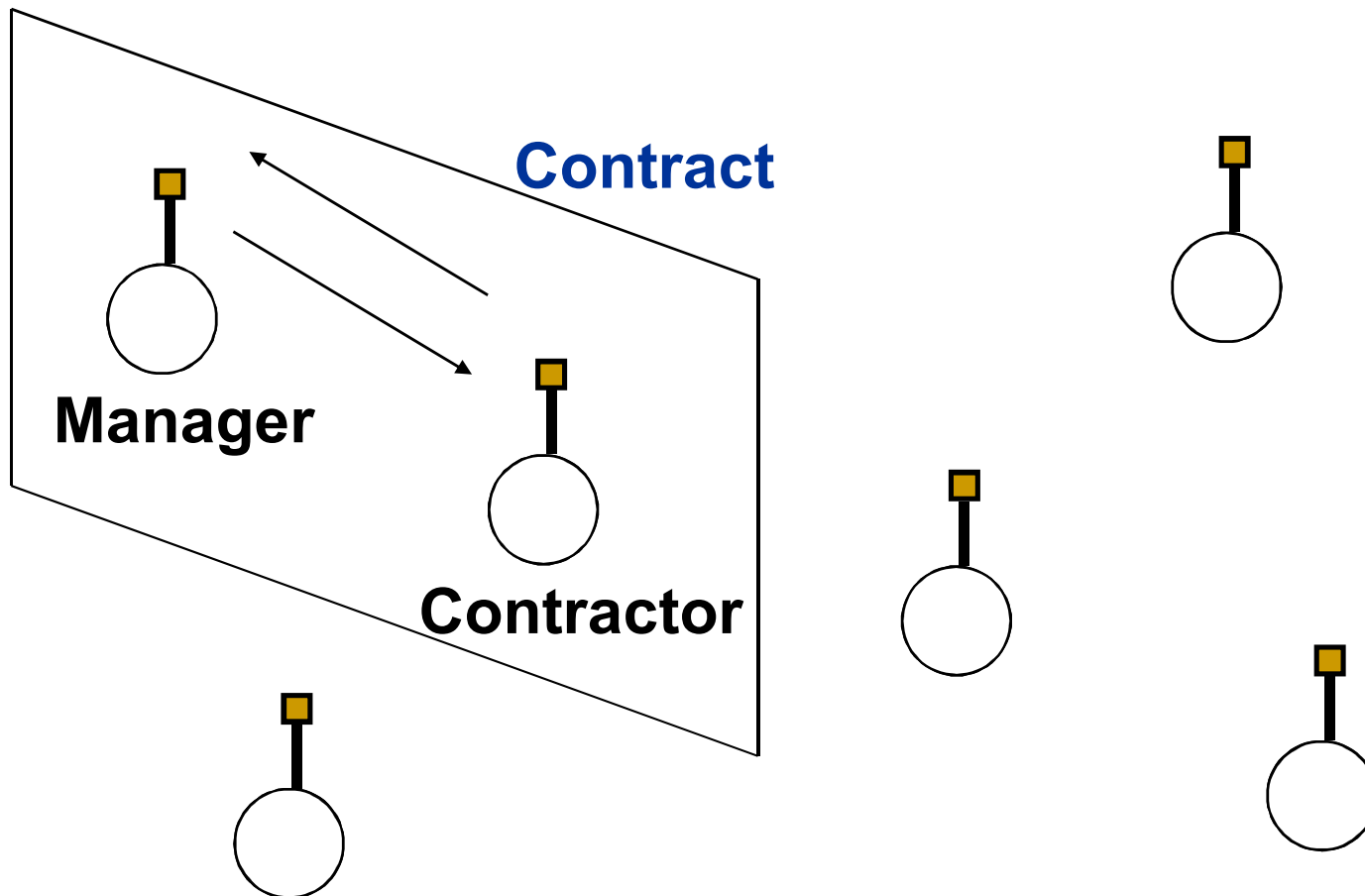


# Manager Making an Award

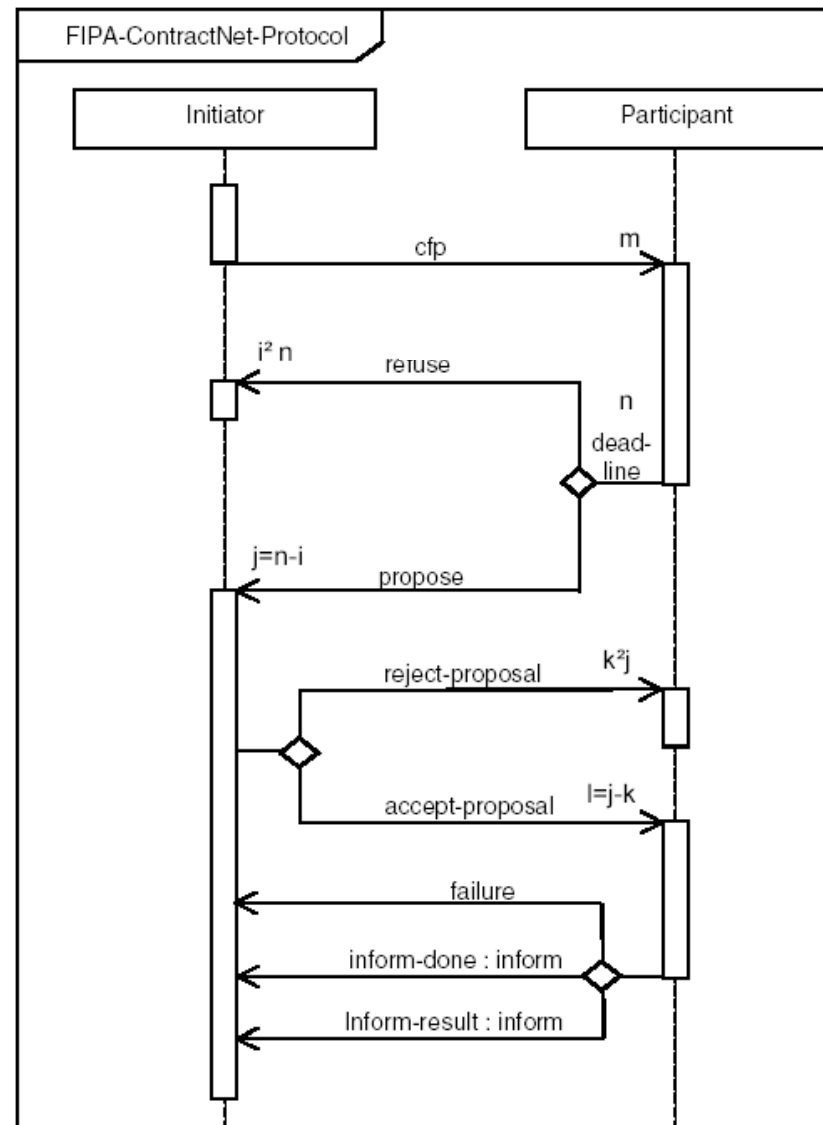




# Contract Established



# FIPA- Contract Net protocol



---

# Contract Net node modules

- Local database

- Knowledge base, info. on the state of negotiations and the state of the solution of tasks

- Interface module

- Sends/receives messages, deals with the communication with the other nodes

- Task processor

- Executes the tasks assigned to the node

- Contract processor

- Studies new offered tasks, submits bids, formalizes contracts
-

---

# Domain-Specific Evaluation

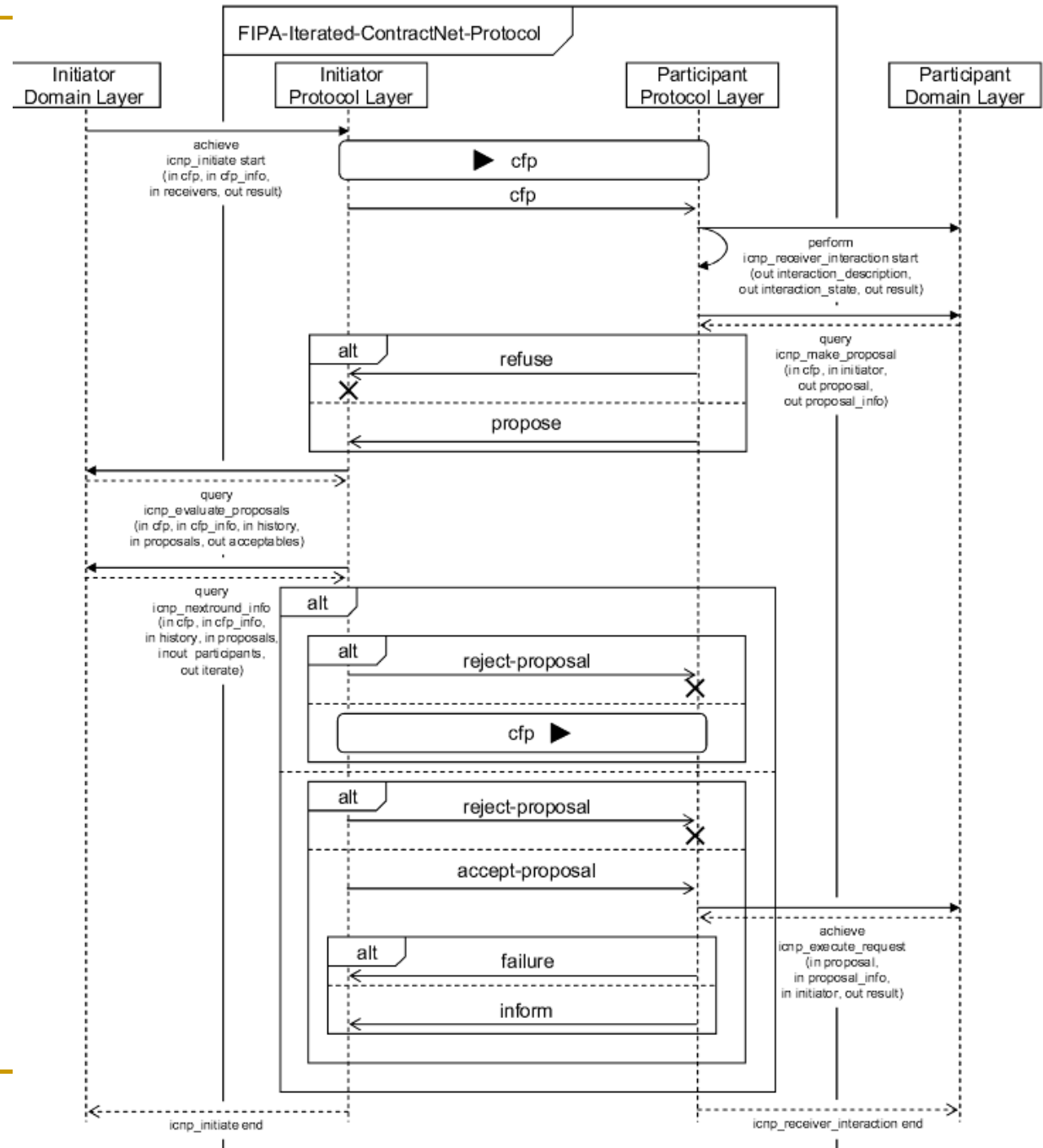
- Task announcement message prompts potential *contractors* to use domain specific **task evaluation** procedures
    - E.g. identifying important attributes
    - There is **deliberation** going on, not just selection — perhaps no tasks are suitable at present
  - *Manager* considers the submitted bids using a domain specific **bid evaluation** procedure
-

---

# Efficiency Modifications

- **Focused addressing** — when general broadcast isn't required
    - Agents could automatically *learn* which are the most appropriate nodes for common tasks
  - **Directed contracts** — when manager already knows which node is appropriate
    - For instance when a very similar task has already been done in the past
  - The nodes can make **proactive offers** to potential managers of the kind of tasks they are able to execute
-

# FIPA- Iterated Contract Net protocol



---

# Features of Contract Net Protocol

- Two-way dynamic transfer of information
  - Mutual selection
    - Bidders select from among task announcements
    - Managers select from among bids
  - Local evaluation
    - Preserving autonomy and private information of agents
-

---

# Suitable Applications

- Hierarchy of Tasks
  - Subtasks are **large** enough (and it's worthwhile to spend effort to distribute them wisely)
  - Primary concerns are **distributed control**, achieving **reliability**, **avoiding bottlenecks**
-



---

# Limitations

- Other stages of distributed problem solving are non-trivial:
    - ❑ Problem Decomposition
    - ❑ Solution Synthesis
  - Computational overhead
    - ❑ Messages
    - ❑ Time – deliberation, analyze offer/bid, wait for decisions
-

---

# What Is An Ontology?

- An **ontology** is an explicit description of a domain
    - ❑ **concepts**
    - ❑ **relationships** between concepts
    - ❑ **attributes** of concepts
    - ❑ **restrictions** on attribute values
    - ❑ **restrictions** on properties of relationships
    - ❑ **individuals** (*often, but not always*)
  - An **ontology** defines
    - ❑ a common vocabulary
    - ❑ a shared understanding of a domain among a set of agents
-

---

# OWL: Web Ontology Language

- The newest **ontology representation language**
    - Since October 2009, **OWL2**
  - Standard worldwide notation
  - Designed to bring semantic content to the Web (**Semantic Web**)
  - WebOnt group developed the OWL formalism
    - OWL language now a W3C recommendation
    - [OWL2 quick reference guide](#)
-

---

# OWL: Language components

- RDF Schemas Features
  - Equality and Inequality
  - Property Characteristics
  - Property Restrictions
  - Logical Operators
-

---

# RDF Schemas Features

- They define **basic ontological components**
    - Classes
    - Subclasses
    - Individuals
    - Properties
    - Subproperties
    - Domain
    - Range
-

---

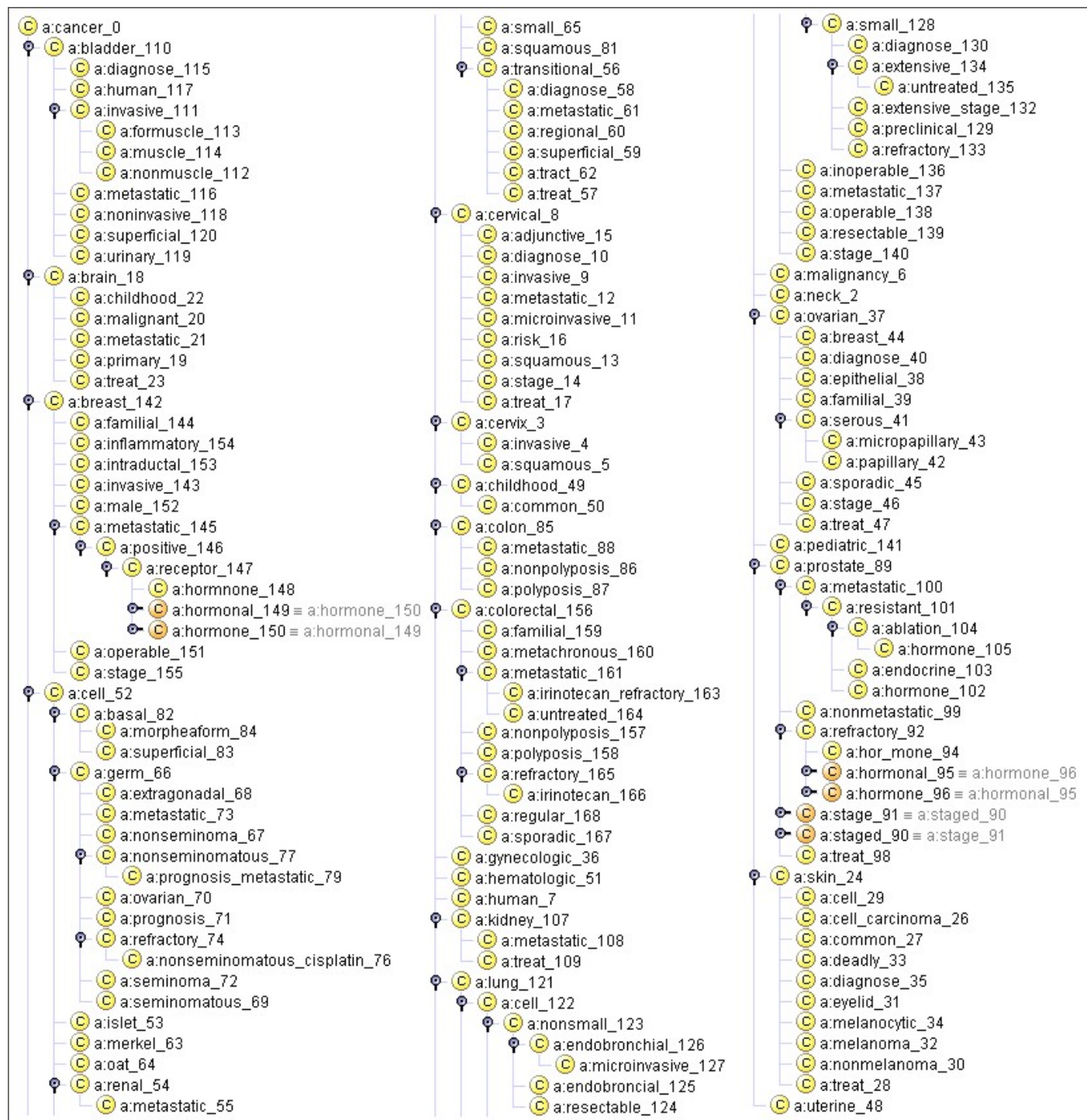
# Classes

- Classes are sets of individuals with common characteristics
  - A Class should be described such that it is possible for it to contain Individuals
  - Classes that cannot possibly contain any individuals are said to be *inconsistent*
  - Eg: Disorder, Patient, Treatment, Symptom
-

---

# Subclasses

- Define **class specializations** by constraining their coverage
  - Ex: **Breast Cancer** is a subclass of **Cancer**
  - **Class hierarchies** can be specified by making one or more statements that a class is a subclass of another class
-





---

# Individuals (Instances)

- Individuals are the **specific objects** in the domain
  - Individuals may be (and are likely to be) members of multiple Classes
  - Ex. **St\_Johns\_Hospital**, **Peter\_Smith\_disorder**
-

---

# Properties

- Properties can be used to state **relationships** between individuals or from individuals to data values
  - Relationships in OWL are **binary**  
Subject → predicate → Object  
Individual a → hasProperty → Individual b  
Individual → hasProperty → Value
  - Eg: hasSymptom, isCausedBy, Author
-

---

# Property types

- ❑ **Object Property**: relates individuals
    - Establishes a relationship between objects
  - ❑ **Datatype Property**: relates individuals to data (int, string, float etc)
    - Can be considered “attributes” of the instance
  - ❑ **Annotation Property**: for attaching metadata to classes, individuals or properties
    - E.g. version, author, comment
-

# Property examples

*Object  
Property*

Disorder  $\xrightarrow{\text{isCausedBy}}$  Cause

*Datatype  
Property*

Disorder  $\xrightarrow{\text{hasScientificName}}$  *String*

*Annotation  
Property*

Disorder  $\xrightarrow{\text{comment}}$  *Meta-data*

---

# Built-in datatypes

## ■ Basic datatypes:

<http://www.w3.org/2001/XMLSchema#name>

- ❑ xsd:string, xsd:boolean, xsd:decimal, xsd:float, xsd:double, xsd:dateTime, xsd:time, xsd:date, xsd:gYearMonth, xsd:gYear, xsd:gMonthDay, xsd:gDay, xsd:gMonth, xsd:hexBinary, xsd:base64Binary, xsd:anyURI, xsd:normalizedString, xsd:token, xsd:language, xsd:NMTOKEN, xsd:Name, xsd:NCName, xsd:integer, xsd:nonPositiveInteger, xsd:negativeInteger, xsd:long, xsd:int, xsd:short, xsd:byte, xsd:nonNegativeInteger, xsd:unsignedLong, xsd:unsignedInt, xsd:unsignedShort, xsd:unsignedByte and xsd:positiveInteger.
-

---

# Sub Properties

- Defines **properties specializations** by constraining their coverage
  - Make hierarchies from one or more statements that a property is a subproperty of one or more other properties
  - Ex. **hasScientificName** is a subPropertyOf **hasName**
-

---

# Domain

- It indicates the individuals to which the property is applied
    - If a property relates an individual A to another individual B, and the property has a class C as its domain, then the individual A must belong to class C
  - Ex. **hasSymptom** has the domain **Disorder**
    - $X \text{ hasSymptom } Y \rightarrow X \text{ is a Disorder}$
-

---

# Range

- It indicates the individuals that receive the application of the property
    - If a property relates an individual A to another individual B, and the property has class C as its range, then the individual B must belong to class C
  - Ex. **hasSymptom** has a range of **Symptom**
    - **X hasSymptom Y → Y is a Symptom**
-



---

# Equality and Inequality

- OWL terms that allow expressing **equalities** and **inequalities** between ontological components
    - **EquivalentClass**: two classes are equivalent
    - **EquivalentProperty**: two properties are equivalent
    - **SameIndividualAs**: different names that refer to the same individual
    - **DifferentFrom**: two individuals are different
    - **AllDifferent**: all members of a list are distinct and pairwise disjoint
-

---

# Property Characteristics

- They define the **semantics** of properties
    - **InverseProperty**: one property is the inverse of another (**hasSymptom, isSymptomOf**)
    - **TransitiveProperty**: the property is transitive (**isCaused**)
    - **SymmetricProperty**: the property is symmetric (**isBrotherOf**)
    - **FunctionalProperty**: the property has a unique value (**Mother**)
    - **InverseFunctionalProperty**: The inverse of the property is functional (**isMotherOf**)
-

---

# Property Restrictions

- Define **constraints** on the use of properties
    - **AllValuesFrom**: all the values in the range of a property belong to a given class
      - Cancer **isTreatedWith** [AllValuesFrom Radiotherapy]
    - **SomeValuesFrom**: at least one value in the range of a property belongs to a given class
      - Flu **hasSymptom** [SomeValuesFrom Fever]
    - **MinCardinality, MaxCardinality**:  
minimum/maximum number of individuals to whom you can be related with a certain property
-

---

# Logical Operators (I)

- Define classes out of other classes
    - IntersectionOf
      - Tuberculosis\_Symptoms = Fever IntersectionOf Coughing\_Blood
    - UnionOf
      - Flu\_Symptoms = Fever UnionOf Vomit
    - ComplementOf
      - StandardDisorder = ComplementOf ContagiousDisorder
-

---

## Logical Operators (II)

- ❑ **DisjointWith**: two classes are disjoint
    - Symptom **DisjointWith** Cause
  - ❑ **OneOf**: defines a class by enumerating all the individuals that belong to it
    - Hospitals is **OneOf** {University-Hospital}, {St\_John}, {City-Clinic}
-

---

# OWL - Conclusions

- OWL is a **language for representing ontologies**, which extends frame languages
  - OWL has a **rich set of features**
  - There exist **reasoners** to check the consistency of an ontology
  - Before building a knowledge base (ontology) an **study of the domain** is required (in order to determine constraints, relationships and incompatibilities)
-

---

## Extra material in Moodle space

- FIPA web page <http://www.fipa.org>
    - FIPA specifications
  - **OWL** official description
  - Link to Protégé web page (ontology editor)
  - Document and presentation on ontology development
  - Presentation on Ontology Languages
  - Video describing OWL
  - Paper on Contract Net (Smith)
-